# Recursion

Part 4

1

# Program Structure

How they work

2

## Program Structure

- When writing a program, you must be aware how it works "behinds the scenes"
- In particular, you must understand memory and how it is used.

## Program Structure

- There are possible issues that can arise that can negatively impact your programs
- … and possibly make them unresponsive

## Some Terminology

- When you call a function, you can specify pieces of data called *arguments*
- These match the format of the function – which is specified in its *parameters*
- Basically
  - arguments are *passed* to the parameters
  - they match, in order, on a one-to-one basis
  - arguments ➔ parameters



## Scope

- *Scope* refers how a variable/function is bound (i.e. visible to the rest of your program)
- Data is often stored differently, based on its scope

# Global Variables

- You can declare variables outside functions
- This are visible to all functions in the class (or module)
- These are known as *global variables*





# Global Variables

```
int total

void printTotal()
{
   print total)
}

int main()
{
   total = 1000
   ...
```

Visible to all functions!

## Global Variables

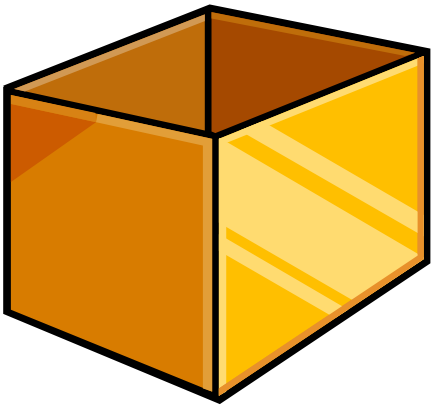
- They can be useful for sharing data between functions
- However, it can be problematic
  - variables can be modified in ways that cause side effects in your program
  - it is better to use local variables and pass them to other functions





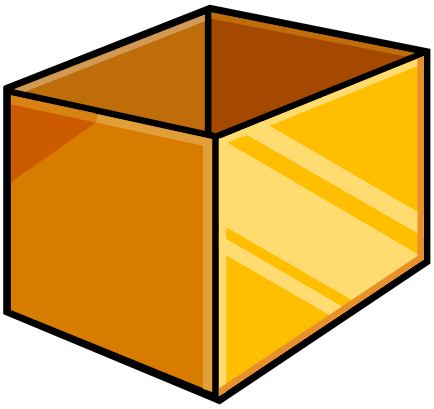
## Local Variables

- When you create functions, each can have *local* variables
- These are only "visible" to the function in which they are declared
- So, other functions cannot access them



# Variable Scope

- Different functions can have local variables with the same name
- Why?
  - they can't "see" each other
  - they are different variables, anyway
  - … so, there is no problem







# Variable Scopes

```
int hello()
{
    int x
}

int main()
{
    int x
}
```

Not the same variable

# Variable Scopes

```
int hello()
{
    double x
}

int main()
{
    int x
}
```

Don't have to be the same type
(they are different variables)

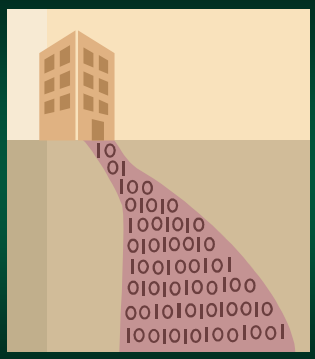




# Example: Average Function

```
double average(double a, double b)
{
    double avg

    avg = (a + b) / 2
    return avg
}
```

Parameters are also local variables



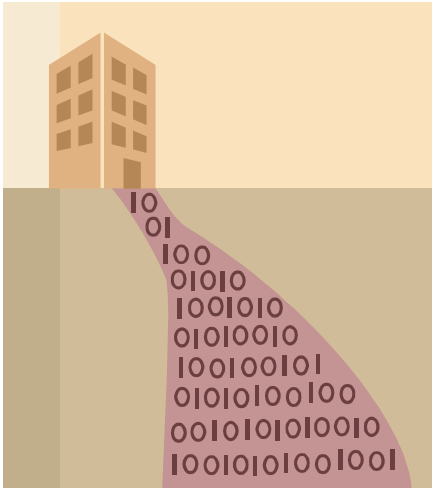
# The System Stack & Heap

Making the Functions Function & Data Delightful

15

## The System Stack & Heap

- Computers maintain two types of memory for running programs: *The Stack* and *The Heap*
- Each has a specific purpose, and, in tandem, they make modern programs possible

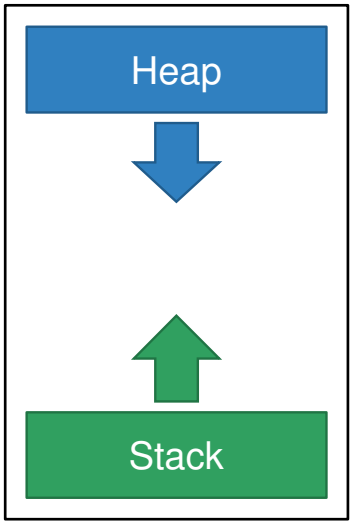




16

## The System Stack & Heap

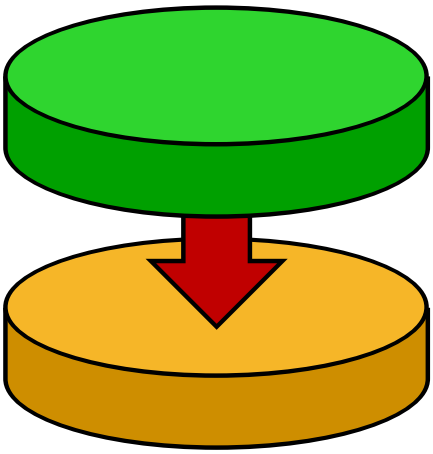
- Each is stored in your computer's main memory
- They grow "towards" each other (and, hopefully, will never meet)





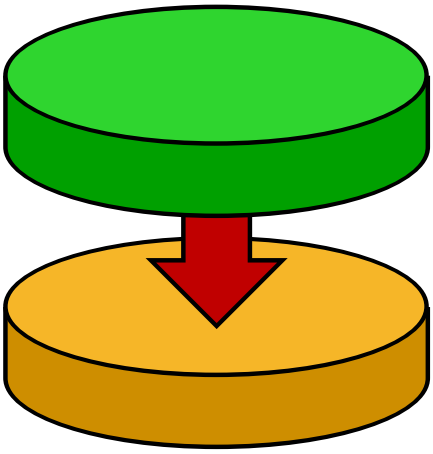

## The System Stack

- The System Stack is used to store local variables and allow your program to support functions
- So, anytime you call a function or declare a local variable, a stack is used



## The System Stack

- Each time a function calls another function an *Activation Record* is placed on the *stack*
- It contains all the information that the instance of a function requires







## Contents of the Activation Record

- The Activation Record contains:
  - parameters
  - local variables
  - return address (used by the processor)
- Data in an activation record is temporary to that "instance" of a function
- In other words, data does not persist after the function ends

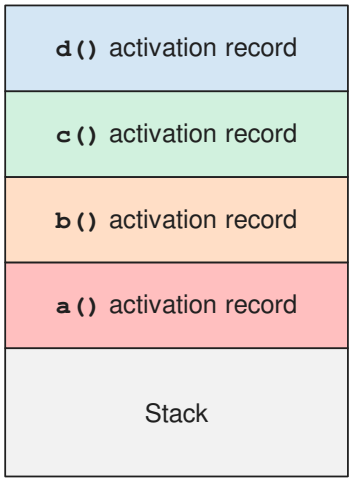
## The Power of Stacks

- Because the stack is a First-In-Last-Out structure, it allows function nesting
- And even a more powerful concept – *recursion*
- Examples
  - web browser "back button"
  - undo sequence in a text editor



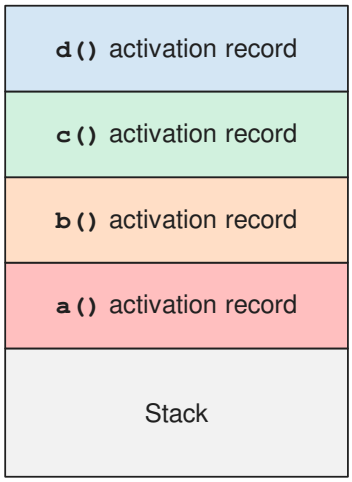
## Nesting Activation Records

- For example:
  - `main()` calls `a()`
  - `a()` calls `b()`
  - `b()` calls `c()`
  - `c()` calls `d()`
- Each activation record is pushed onto the stack



# Nesting Activation Records

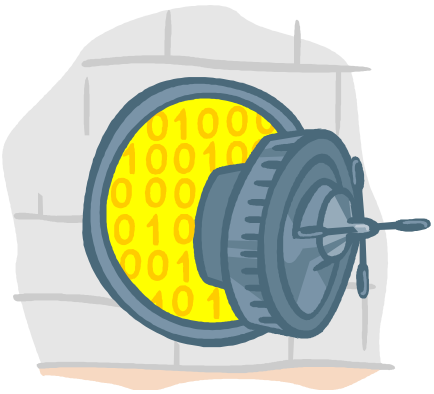
- When a function "returns", its activation record is pop'd and discarded
- The local variables cease to exist
- Only the return value is passed to the caller





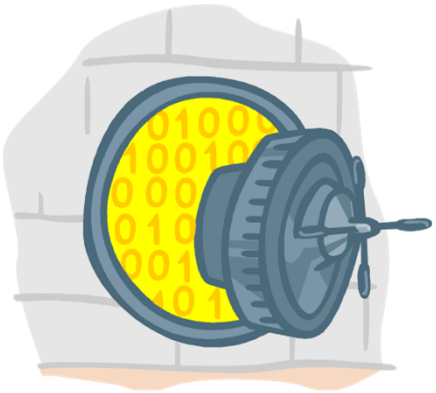

# The Heap

- Nothing on the system stack persists forever – it is quite temporary
- So, how do we make data last indefinitely? …or, as long as our program is active



## The Heap

- *The Heap* is used to store *dynamic* allocation
- It is allocated *as needed*
- … not to be confused with the Heap Data Structure (which we will cover later)





## The Heap

- Anytime you create objects using *"new"*…
  - the heap is used to allocate storage
  - system performs garbage collection after the memory is no longer needed
- Unlike the stack, data persists regardless of function calls

# Reference Types

The objective of Object Oriented Programs



## Reference Types

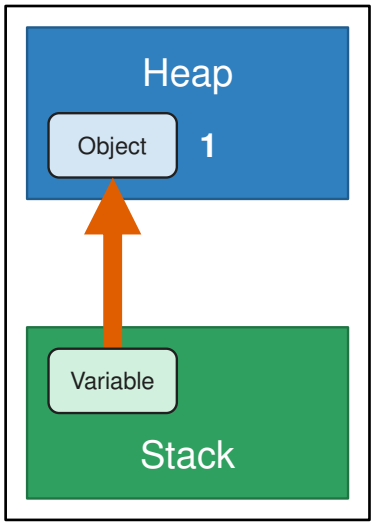
- Most languages are based on largely based on building abstract data types called *reference types*
- They are links to nebulous *objects* – whose contents & implementation are unknown

## Reference Types

- This is essential for object-oriented programming to work
- … and is the basis of all modern programming languages

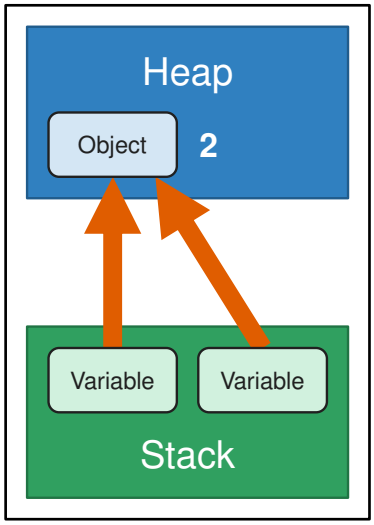
## Reference Types

- So, local variables exist on the stack
- But... they reference *(contain the address of)* objects stored on the system heap



# Reference Types

- This allows multiple variables to point to the same object
- This is called *aliasing*
- The system keeps track of how many references each object has








# Example Aliasing

```
Student a, b

a = new Student("Joe Gunchy", "CSc")
b = a
```

**b** and **a** reference the same student

## Garbage Collection

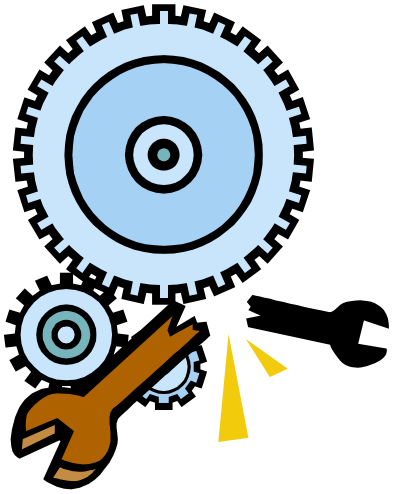
- Programming languages use *garbage collection* reclaim unused data from the heap
- Policy is to reclaim the memory used by objects that *can no longer be accessed (i.e. **no** references)*

## Garbage Collection

- So, languages maintain a counter on each object
  - if you add a reference, it increments
  - if a reference is removed, it decrements
- When it reaches zero, the object can be removed

## Loitering

- It is possible to "remove" an item from the ADT, but accidently keep a reference (link) to it
- The item is effectively an *orphan* - it will be never be accessed again by the ADT

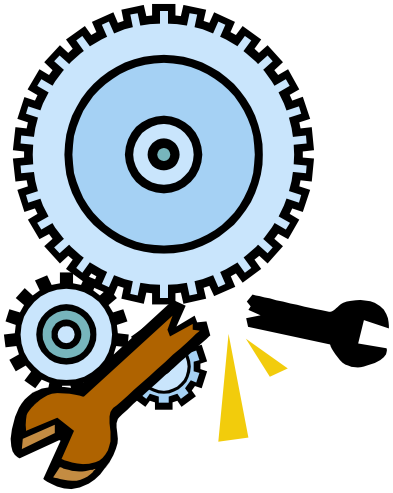




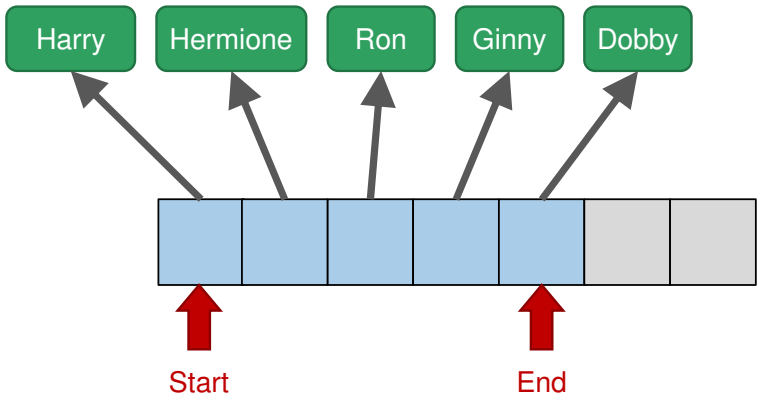


## Loitering

- The garbage collector has no way to know unless it's overwritten
- So, under this condition, the object is said to *loiter* – stay in memory with no purpose
- This can negatively affect performance



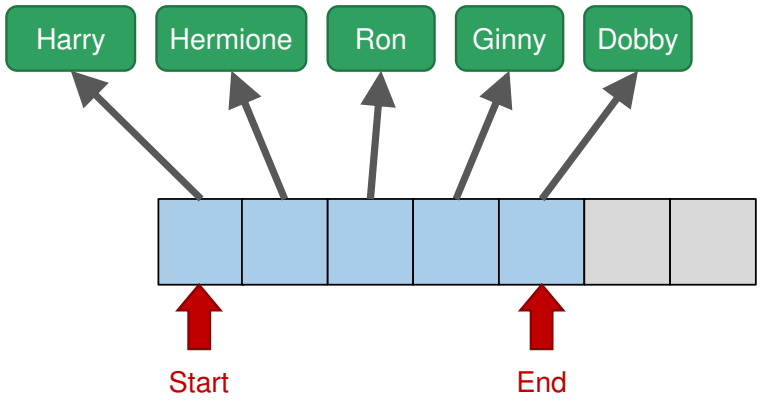
# Array Storing a List (partially filled)



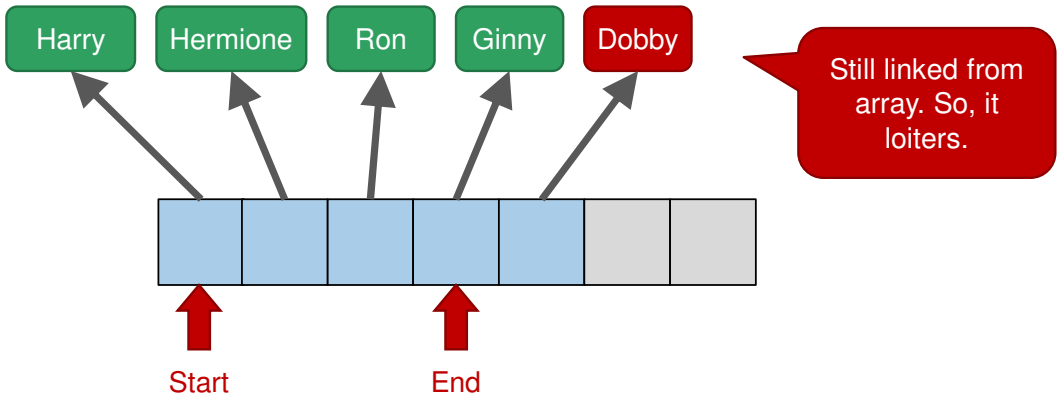





# Delete Last – Move End



# Dobby is still linked…



# Pools

Okay, now it's getting weird

## Pools

- Creating and destroying objects is expensive on the heap
- So, we want to minimize the constant creation and deletion of new nodes

## Jump in the Pool

- One solution is to maintain a *pool*
- This is a collection of nodes that are allocated early and are used as, kind of, a recycling bin

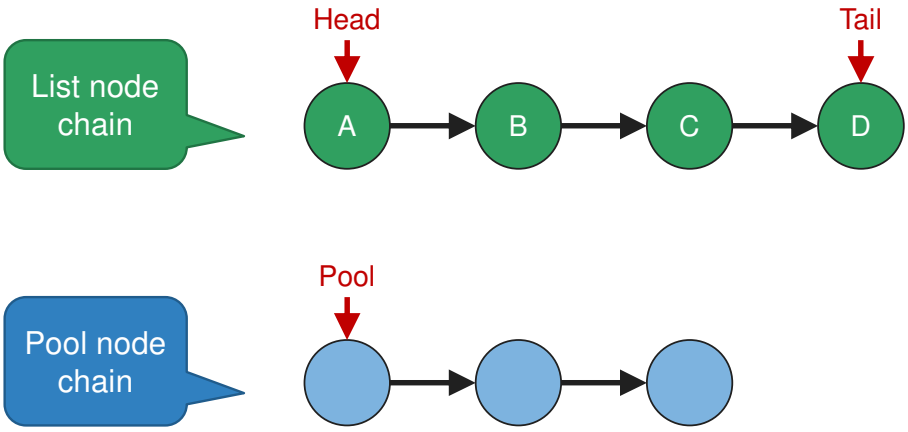
## Jump in the Pool

- If a node is needed, one is removed from the pool
- If a node is removed, and the array has room, it is placed back in the array (after the data field is set to null, of course)

## Even more approaches

- You can also use a "pool" for linked lists
- So, your Linked List class
  - would have a linked list of valid nodes
  - and another list of unused nodes
  - the danger here is that you don't limit the size of the pool – and it grows ***forever***
  - so, if you use two linked lists, keep a pool member count

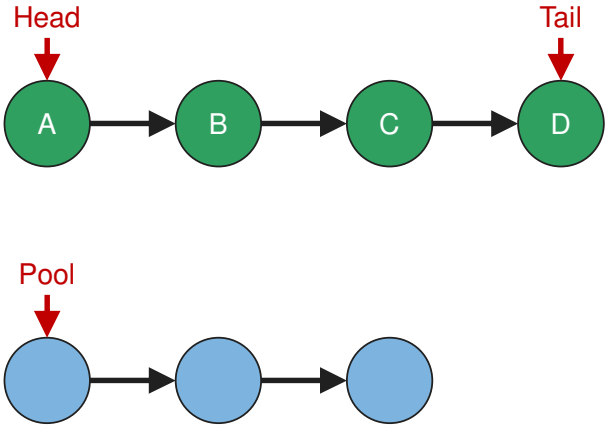
# Linked List with a List Pool



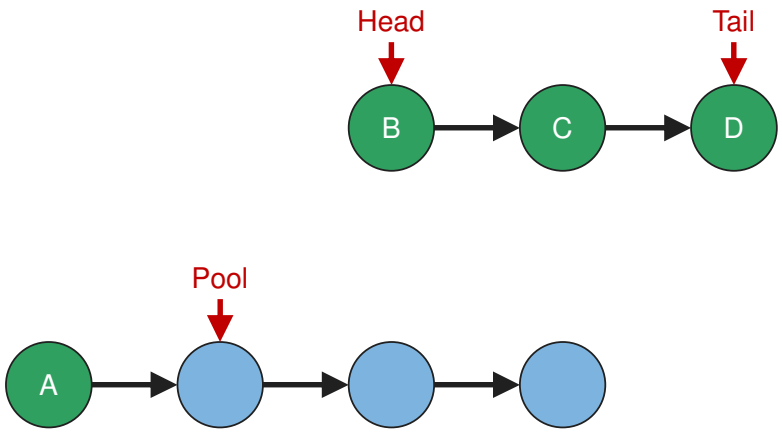





# Delete Head: Remove Node (1 of 3)



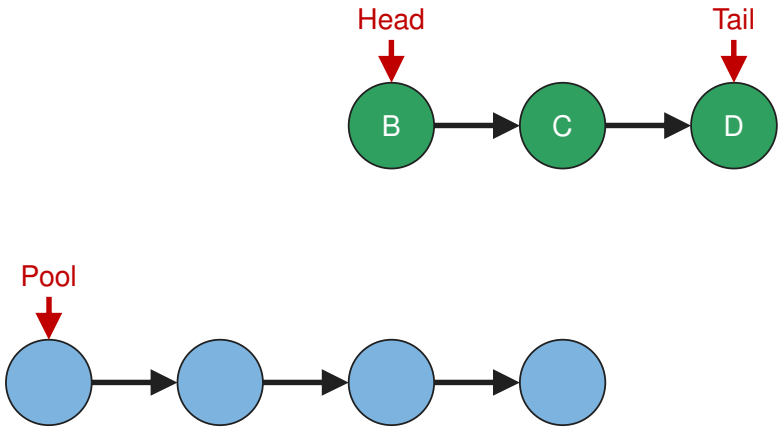
# Delete Head: Link (2 of 3)



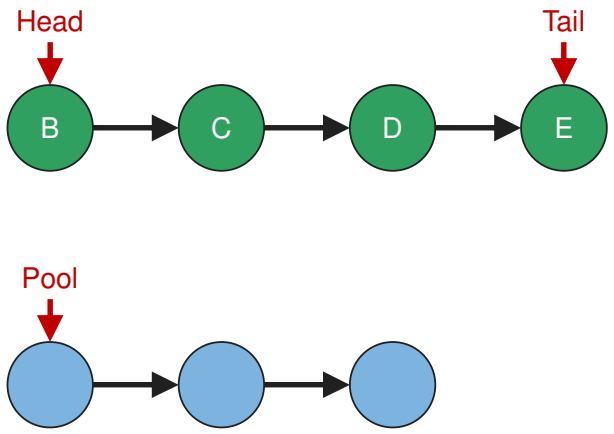





# Delete Head: Clear Value (3 of 3)



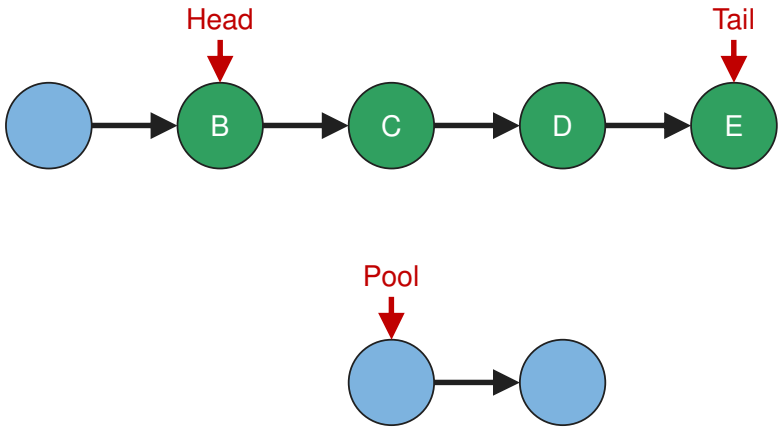
# Add Head: Remove from Pool (1 of 3)



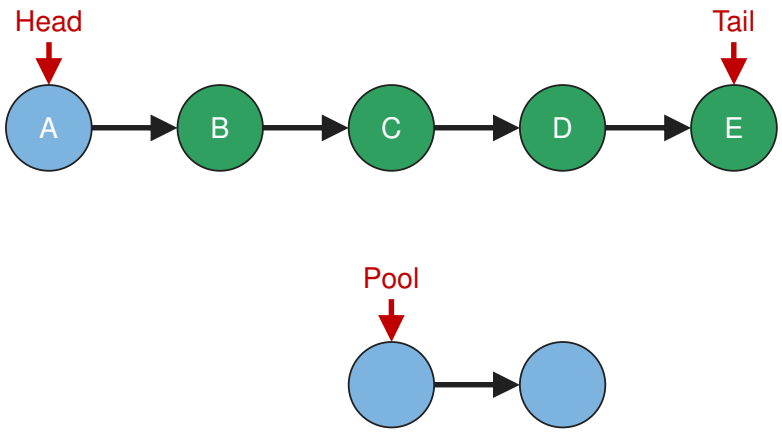





# Add Head: Link (2 of 3)



# Add Head: Set Value (3 of 3)

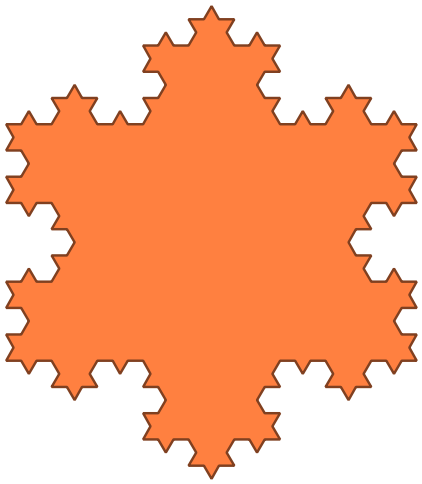


# Recursion

The best way to learn recursion…
is to, first, learn recursion!

## Recursion

- *Recursion* occurs when a function directly or indirectly calls *itself*
- This results in a loop
- However, it doesn't use iterative structures such as For or While loops

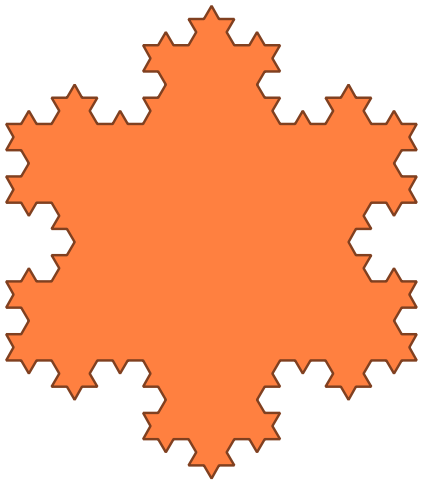






## Recursion

- This can greatly simply programming tasks
- Commonly used to traverse a graph, tree, or run complex calculations
- While powerful, it is costly on computer resources
- …and can also create pitfalls



## Some Well-known Problems

- Sorting
- Searching
- Shortest paths in a graph
- Minimum spanning tree
- Primality testing

## Some Well-known Problems

- Traveling salesman problem
- Knapsack problem
- Chess
- Towers of Hanoi
- Program termination

## Breaking a Problem Down

- Recursion allows a problem to be broken down into smaller instances of themselves
- Each call will represent a smaller, simpler, version of the same problem
- Eventually, it will reach a *"base case"* which will not require any more recursive calls



## Where Recursion Shines

- When the program can be broken into smaller pieces, recursion is a great solution
- Examples:
  - graph traversal – searching, etc….
  - state machines
  - sorting
  - many math problems

## Danger: Never Ending

- If you break down a task into smaller parts… at some point, it should become a single value
- If not, the function will never end and will recurse forever – *at least until the computer runs out of resources*

## Danger: Accidental Recursion

- Accidental recursion is a common mistake by beginner programmers
- Recursion can be done directly or indirectly
  - for example: A calls B, B calls C, C calls A
  - organize your code carefully!

## Results of These Dangers…

- Runaway recursion
  - function will recurse ***forever***
  - eventually all memory is exhausted
- You will see either…
  - "stack overflow" error
  - "heap exhaustion" error

## To infinity… but not beyond

```
function toInfinity()

   print "To infinity!"
   toInfinity()
   print "and beyond!"

end function
```

We never get here!

# Designing a Recursive Function

- Does the problem lend itself to recursion?
  - can the problem be broken down into smaller instances of itself?
  - is there a iterative version that is better
- Is there a base case?
  - is there a case where recursion will stop?
  - remember: ALWAYS have a stopping point!

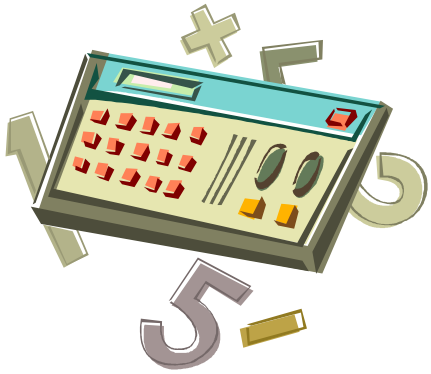
# Examples of Recursion

Examples defined as examples defined as examples…



## Example1: Factorials

- Factorials are classic mathematical problem that lends itself easily to recursion
- If you don't remember, a factorial of *n* is defined as the value of n multiplied by all lesser integers ≥ 1
- Eg: 5! → 5×4×3×2×1 → 120



## Example 1: Factorials

- It should be easy to observe that *n!* can be defined as $n \times (n - 1)!$
- So, n! can be computed by multiplying n by the factorial of one less than it
- $4! \rightarrow 4 \times 3! \rightarrow 4 \times 3 \times 2! \rightarrow 4 \times 3 \times 2 \times 1$





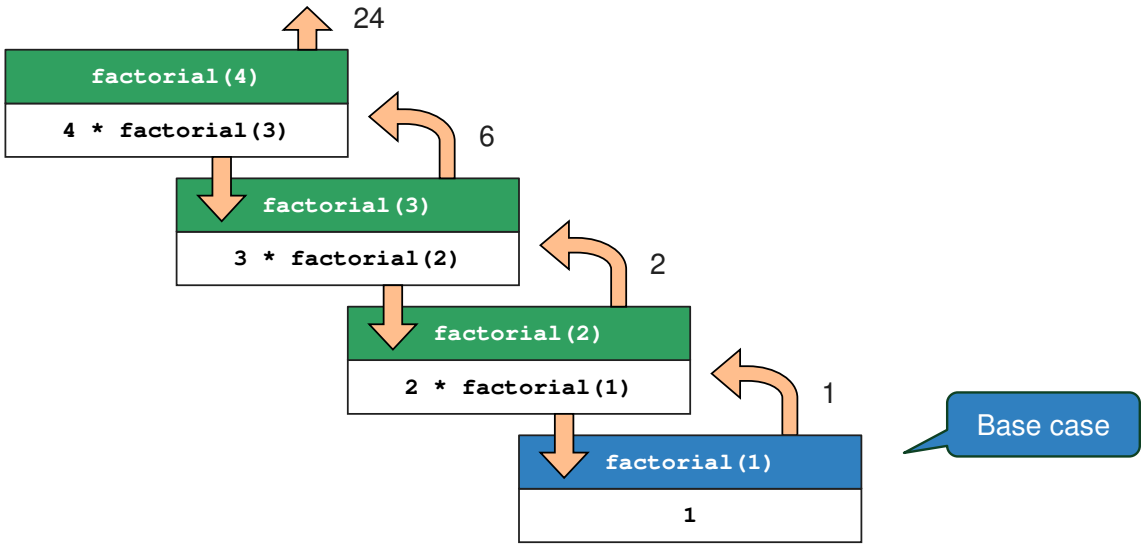
## Example 1: Factorials

```
function factorial(int n) returns int

    if (n == 1)
       return 1
    else
       return n * factorial(n - 1)
    end if

end function
```

Base case

Recursive case

# Example 1: Factorial








# Example 1: Factorials

```
public function main()

    print factorial(4)
    print factorial(7)
    print factorial(12)

end function
```

# Example 1: Output

```
24
5040
479001600
```

Yes, it grows quickly!





# Example 2: Quagmire

- Glen Quagmire is a character on the show Family Guy
- Besides his (almost illegal) antics, he is known for his catch phrase "Giggity goo!"
- The number of times he says "giggity" varies depending on the situation

# Example 2: Quagmire

- We can solve this recursively
- If we look at "giggity giggity goo!", we can observe that it is "giggity" + "giggity goo!"
- We can print his catch phrase using recursion.

# Example 2: Quagmire method

```
function quagmire(int count)

    if (count == 0)
        print "goo!"
    else
        print "giggity "
        quagmire(count - 1)
    end if

end function
```

Base case

Recursive case

# Example 2: Quagmire – return a String

```
function quagmire(int count) returns String

    if (count == 0)
        return "goo!"
    else
        return "giggity " + quagmire(count - 1)
    end if

end function
```

Base case

Recursive case

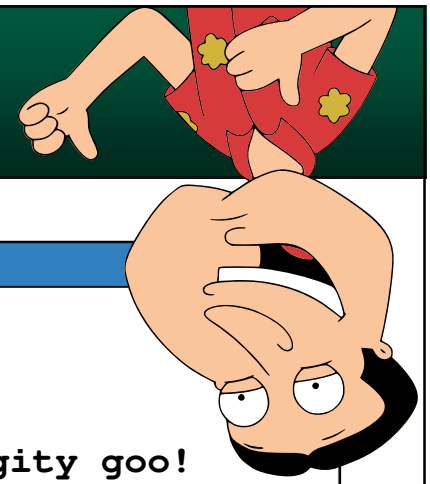




# Example 2: Quagmire

```
public function main()

    print quagmire(1)
    print quagmire(2)
    print quagmire(5)

end function
```

## Example 2: Output



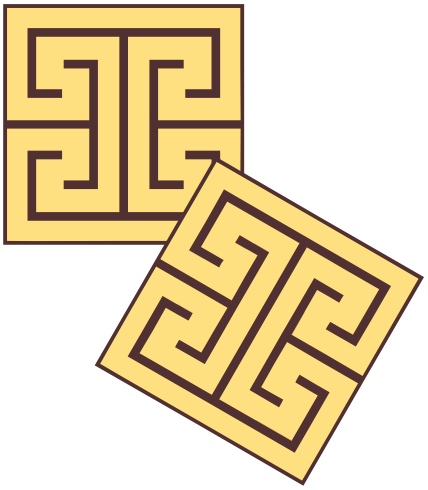
```
giggity goo!
giggity giggity goo!
giggity giggity giggity giggity giggity goo!
```





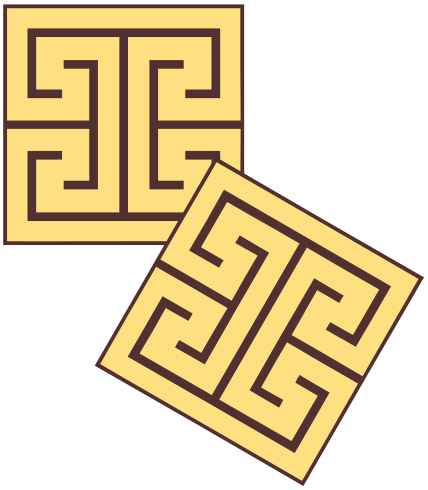
## Example 3: Greatest Common Divisor

- *Euclid* created an ingenious algorithm for finding the greatest common divisor
- This is known example of recursion – first solved using geometry using the metaphor of a tile floor



## Example 3: Greatest Common Divisor

- A common problem in computer science is finding the greatest common divisor or two integers
- For example:
  the GCD of 64 and 40 is 8





## Example 3: Euclid's Algorithm

- Euclid's algorithm is recursive
- You reapply the expression below until the second value of **`gcd(n, m)`** is zero.
- In this case, n will be the GCD

```
gcd(n, m) → gcd(m, n mod m)
```

## Example 3: Greatest Common Divisor

- 60 and 24
  - gcd(60, 24) → gcd(24,12) → gcd(12, 0)
  - the result is 12
- 84 and 20
  - gcd(84, 20) → gcd(20, 4) → gcd(4, 0)
  - result is 4
- These might seem trivial, but it can find HUGE numbers quite easily





## Example 3: Greatest Common Divisor

```
function gcd(int n, int m) returns int
    if (m == 0)
        return n
    else
        return gcd(m, n % m)
    end if
end function
```

Base case

Recursive case

## Example 3: Greatest Common Divisor

```
public function main()

   print gcd(10, 95)
   print gcd(187, 51)
   print gcd(240, 36)

end function
```



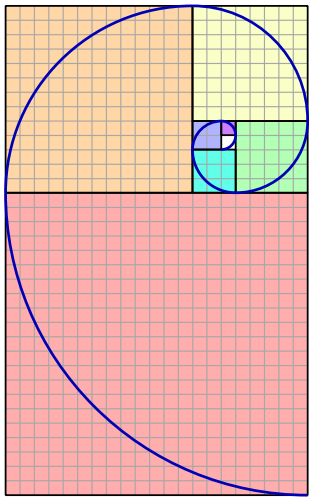


## Example 3: Output

```
5
17
12
```

## Example 4: Fibonacci Numbers

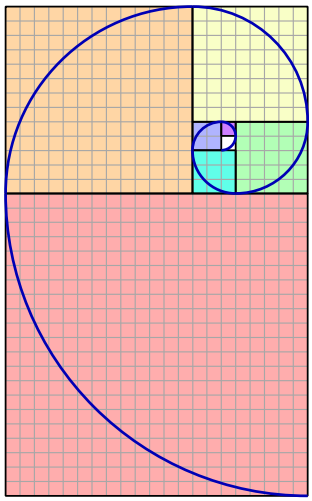
- Rabbits tend to reproduce like… well… rabbits
- Mathematician *Fibonacci* analyzed this situation and created a mathematical system to predict this phenomena



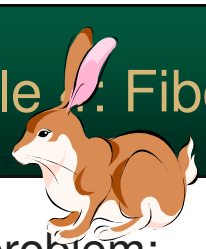


## Example 4: Fibonacci Numbers

- It is used today in finance, simulation, and several computer science algorithms
- As you get see with the picture, it seems to be built into nature itself



## Example 4: Fibonacci Numbers



- The problem:
  - start with a pair of rabbits
  - at month #2, the rabbits begin to reproduce
  - the female gives birth to a new pair of rabbits: one male and one female
  - babies mature at the same rate and will have more babies
- Fibonacci number sequences predict the total pairs after *n* months





## Example 4: Fibonacci Numbers

- After two months, the female gives birth creating a new pair…. then they get pregnant again!
- This continues forever…..
- Sequence: 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

```
if n == 1 then Fib(n) = 1
if n == 2 then Fib(n) = 1
if n > 2  then Fib(n) = Fib(n - 2) + Fib(n - 1)
```

# Example 4: Fibonacci Numbers

```
f(3) = f(2) + f(1) = 1 + 1 = 2

f(4) = f(3) + f(2) = 2 + 1 = 3

f(5) = f(4) + f(3) = 3 + 2 = 5

f(6) = f(5) + f(4) = 5 + 3 = 8
```





# Example 4: Fibonacci Numbers

```
function fibonacci(int n) returns int

    if (n <= 2)
        return 1
    else
        return fibonacci(n - 2) + fibonacci(n - 1)
    end if

end function
```

Recursion:
2 different paths!

## Example 4: Fibonacci Numbers

```
public function main()

   print fibonacci(1)
   print fibonacci(8)
   print fibonacci(12)

end function
```





## Example 4: Output

```
1
13
89
```

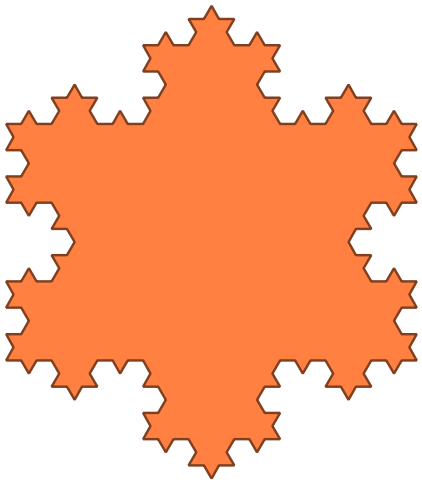
# Recursion in Data Structures

Some data structures are built for recursion

93

## Recursion in Data Structures

- Many data structures can be recursively defined
- In other words, the data structure itself can be broken in smaller versions of itself

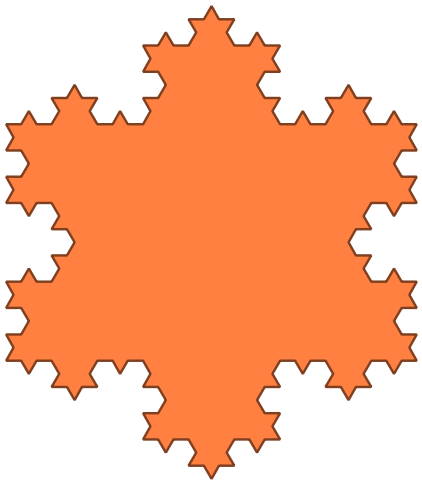




94

## Recursion Example: Sum

- Recursion usually happens in the recursively defined structure itself
- In other words, whatever the structure is made of, that unit will handle recursion



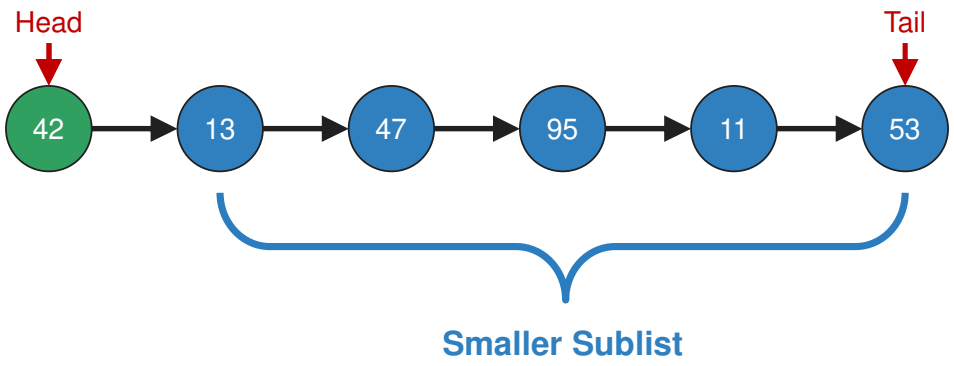




## Example: Linked Lists

- Linked Lists can be recursively defined
- Every list can be seen as collection of smaller lists
- Since Linked Lists are made of nodes, the node class will handle recursion.

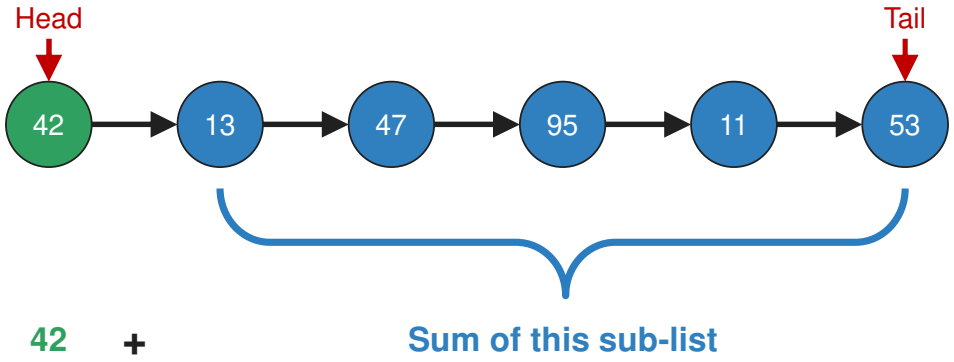
# Recursive Sum



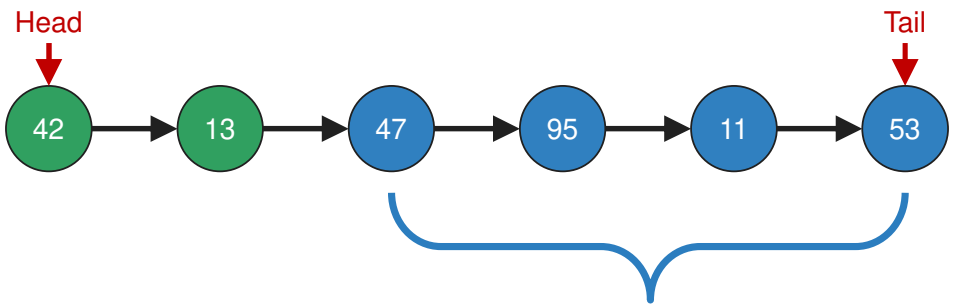





# Recursive Sum



# Recursive Sum

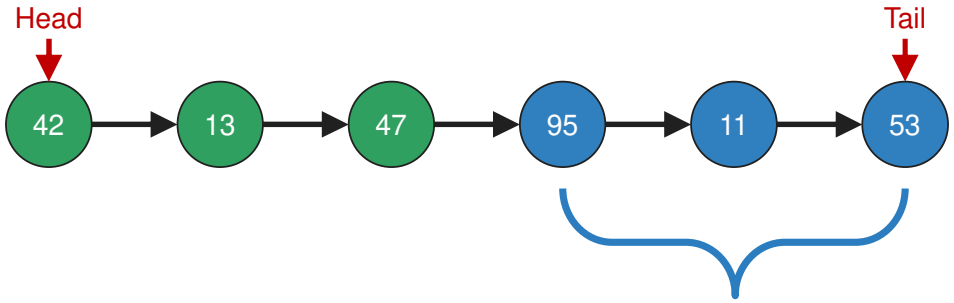



42 + 13 + Sum of this sub-list





# Recursive Sum




42 + 13 + 47 + Sum of this sub-list

# Sum Example (in the node class)

```
function sum() returns double

    if (this.next == null)
        return this.value
    else
        return this.value + this.next.sum()
    end if

end function
```

Base case
(end of the list)

Recursion on smaller list